GraphRAG Explorer

Who owns whom in Austrian banking, and how do you find out?

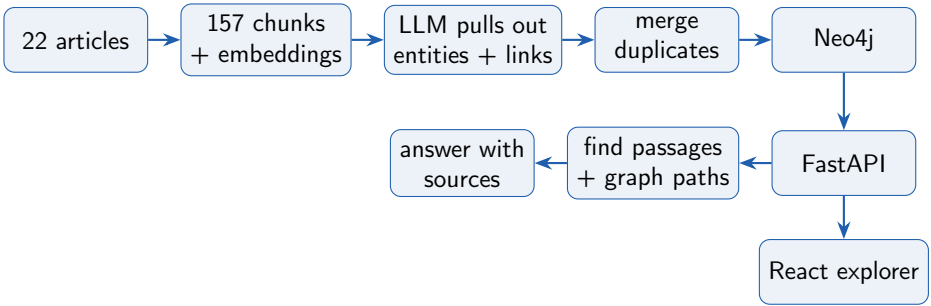
Kacper Zaleski

What I wanted to build

- An app where you can **click through** how banks, owners and regulators are linked, and stumble on things you didn't know
- Every fact comes with **the sentence it was taken from**
- And one honest question: does the graph actually give **better answers** than plain RAG?

Data	22 Wikipedia articles about Austrian banks
Stack	Python, FastAPI, Neo4j, React + Cytoscape.js, Docker
LLMs	Gemini free tier; Groq, Cerebras or Mistral with one setting

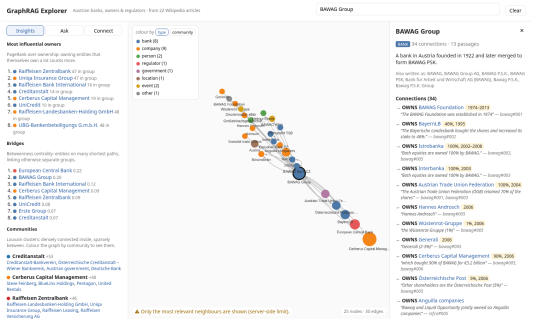
How it fits together



- The result: **726 entities and 585 links**, each one pointing back to its source text
- Running the import again costs nothing: every LLM call is cached

Let me show you

1. Where to start: the biggest owners and groups
2. BAWAG: who owned it, when, and how much
3. How is ÖGB linked to GoldenTree?
4. How is Refco linked to Cerberus?
5. Ask: *Who have been the owners of BAWAG over time?*
6. Ask: *How is BayernLB connected to Addiko Bank?*



Turning text into a graph

- The LLM reads each chunk and fills in a **fixed form**: who's mentioned, how they're linked
- Only 8 kinds of things and 9 kinds of links, so the output stays comparable
- "A is a subsidiary of B" and "B owns A" end up as **the same link**
- The LLM has to **quote its evidence**, and the code checks the quote is really in the text
- One bad link gets dropped, not the whole chunk

What went wrong: my first check wanted the exact quote and threw away 73 links. About 70 of them were fine, the model had just shortened the quote with "…". Checking word by word fixed it, and it still catches the model mangling umlauts ("Österreichische" → "sterr ischische").

Is "ÖGB" the same as "Austrian Trade Union Federation"?

- Simple rules first: case, accents, ö→oe, "AG", "GmbH" and so on
- Everything else goes in a **short alias list that a person checked**
- When in doubt, I **keep things apart**: wrongly merging two Raiffeisen banks would invent an owner that doesn't exist

What went wrong: the most important chain, ÖGB → BAWAG ← Cerberus, was broken because the LLM sometimes wrote the short names and sometimes the long ones. Fun fact: Wikipedia itself spells it "Ceberus" once.

Answering a question with the graph

1. Search the text two ways, by meaning and by keywords, and **combine the rankings**
2. Spot the banks and companies the question names
3. Find the **shortest path** between them, then look around their neighbourhood
4. Skip places: "both are in Vienna" isn't a useful connection
5. Pick the best passages, and **always keep the ones** that prove the path
6. Answer with sources. If the model cites something it never saw, the code removes it

What went wrong: I first blocked very connected nodes so paths wouldn't go through hubs. But here the well-connected banks *are* the answer. Skipping places was what actually worked.

What makes exploring interesting

- **When and how much**: a second LLM pass adds years and stakes to about 200 ownership links
  BayernLB 46% (1995) → ÖGB 100% (2004) → Cerberus 90% (2006) → GoldenTree 21.8%
- **Where to start** when you know nothing:
  - Who owns the most (PageRank over ownership)
  - Who connects otherwise separate groups (ECB, BAWAG)
  - Which banks belong together (Erste with its CEE banks, UniCredit with Bank Austria)
- Finding a path needs no LLM, so it's instant and free

What went wrong: a card-payments company called SIA showed up as one of the five biggest owners. "UniCredit sold its card business to SIA" had been read as "SIA bought UniCredit". One wrong link was enough to throw off the whole ranking.

Living with free-tier APIs

- APIs were slow, overloaded or out of quota, often
- So: timeouts, retries, my own **rate limiter**, and a **backup model** when one fails
- Every answer is **cached**: re-runs are free and give the same result
- If an import fails halfway, nothing is written, and the next run picks up where it stopped

What went wrong: the model I wanted allows **20 requests a day**. The import needs 160. And for embeddings, each text in a batch counts as its own request.

How I tested it

- 28 questions: 13 that need several steps, 10 simple lookups, 5 about the whole collection
- Questions written from the **articles, not from the graph**, so the graph can't grade itself
- An LLM grades the answers, but **from a different company** than the one answering, since models tend to like their own style
- A second grader checks the first one
- Both modes answer the **same questions**, so I compare them question by question

Round 1: a tie

	plain RAG	graph
score [95% CI]	0.91 [0.80, 0.98]	0.91 [0.84, 0.98]
correct	24/28	23/28
right passages found (recall@8)	0.90	0.89
answer time / tokens	1.7 s / 2.8k	2.5 s / 3.4k

- No difference at all (+0.00, McNemar p = 1.0)
- The questions were **too easy**: they name everything, so plain search finds it too
- Both fail on "list all . . ." questions: 8 passages can't hold a full list

Round 2: harder questions, prediction written down first

- My bet, **written down before running**: the graph wins when the link in between is never named
- 8 questions like "How is A connected to C?"
- I didn't touch anything after seeing the results

	plain RAG	graph
score	0.38	0.44
correct	3/8	2/8
right passages found	0.56	0.50
... **without the reranker**	**0.81**	0.56

- $+0.06$, but anywhere from -0.12 to $+0.25$: **my bet didn't hold up**
- The biggest effect was a surprise: the **reranker throws away** the passages plain RAG needs
- The original 28 still work (graph 0.93, plain RAG 0.91), and the two graders agree on 64 of 72 answers ($\kappa = 0.62$ / 0.69)

What I took away

Where the graph is worth it

- Exploring: paths, neighbours, rankings
- Trust: every link shows where it came from
- Time and stakes on ownership
- Mistakes are easy to see (SIA, Cerberus)

Where it isn't (yet)

- Better answers on a small, clean set of articles
- It only knows what was extracted: "Addiko was spun off from Hypo" never made it in
- "List all..." needs a graph query, not a search

"No difference" is a real result, as long as it's measured properly.

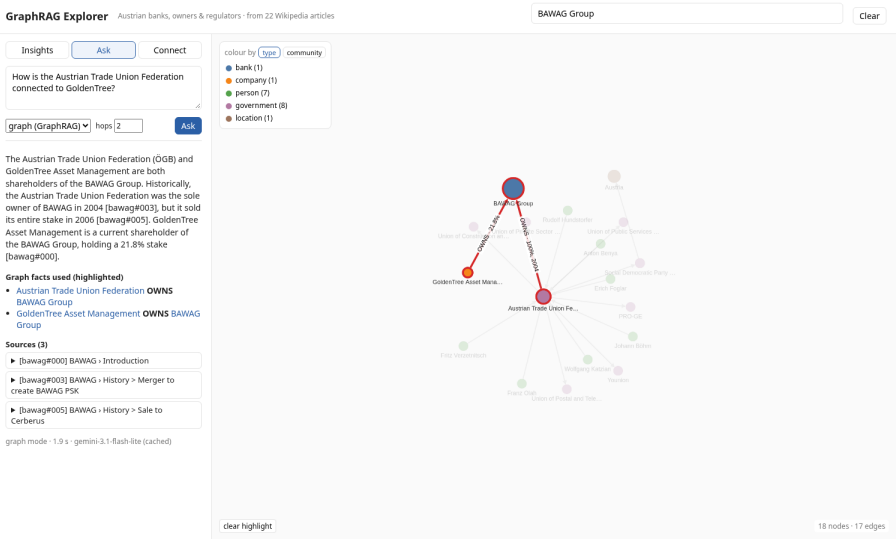
What I'd do next

- Drop the reranker in graph mode (it was trained on web searches, not on questions like these)
- Answer "list all…" questions with a graph query instead of a search
- Read chunks with few links a second time, to miss fewer facts
- A timeline slider in the explorer (the years are already in the data)
- In a bank: models hosted in the EU or in-house, Snowflake as the source, Airflow to re-import on a schedule

Thanks! Questions?

Code, evaluation and notes are in the `github.com/kacperzaleski/graphrag-explorer` repository

Backup: an answer with its path highlighted



Backup: finding a path

